计算机组成原理

指令系统

考试大纲

四、指令系统

（一）指令系统的基本概念

（二）指令格式

（三）寻址方式

（四）数据的对齐和大/小端存放方式

（五）CISC 和 RISC 的基本概念

（六）高级语言程序与机器级代码之间的对应

1.编译器，汇编器和链路器的基本概念

2.选择结构语句的机器级表示

3.循环结构语句的机器级表示

4.过程（函数）调用对应的机器级表示

指令与指令集体系结构(ISA)

机器指令 (Machine Instruction)

- **定义：**计算机硬件能够直接识别并执行的最小功能单位。
- **指令字 (Instruction Word)：**表示一条指令的一位二进制代码串。
- **指令集 (Instruction Set)：**一台计算机所能执行的所有机器指令的集合。

指令集体系结构 (ISA)

- **定位：**软硬件交界界面（Hardware/Software Interface）。
- **规范内容：**
  - 指令格式、操作码定义、寻址方式
  - 通用寄存器个数、字长、程序计数器(PC)
  - 内存编址方式、字节序（大端/小端）
  - 中断与异常处理机制

概念辨析：ISA属于**计算机体系结构 (Architecture)** 范畴，确定了“计算机能做什么”；而指令的具体电路实现（如微程序/流水线）属于**计算机组织 (Organization)** 范畴，解决“如何实现”。同一种ISA可以有不同的硬件实现（如Intel Core i7与AMD Ryzen均实现x86-64 ISA）。

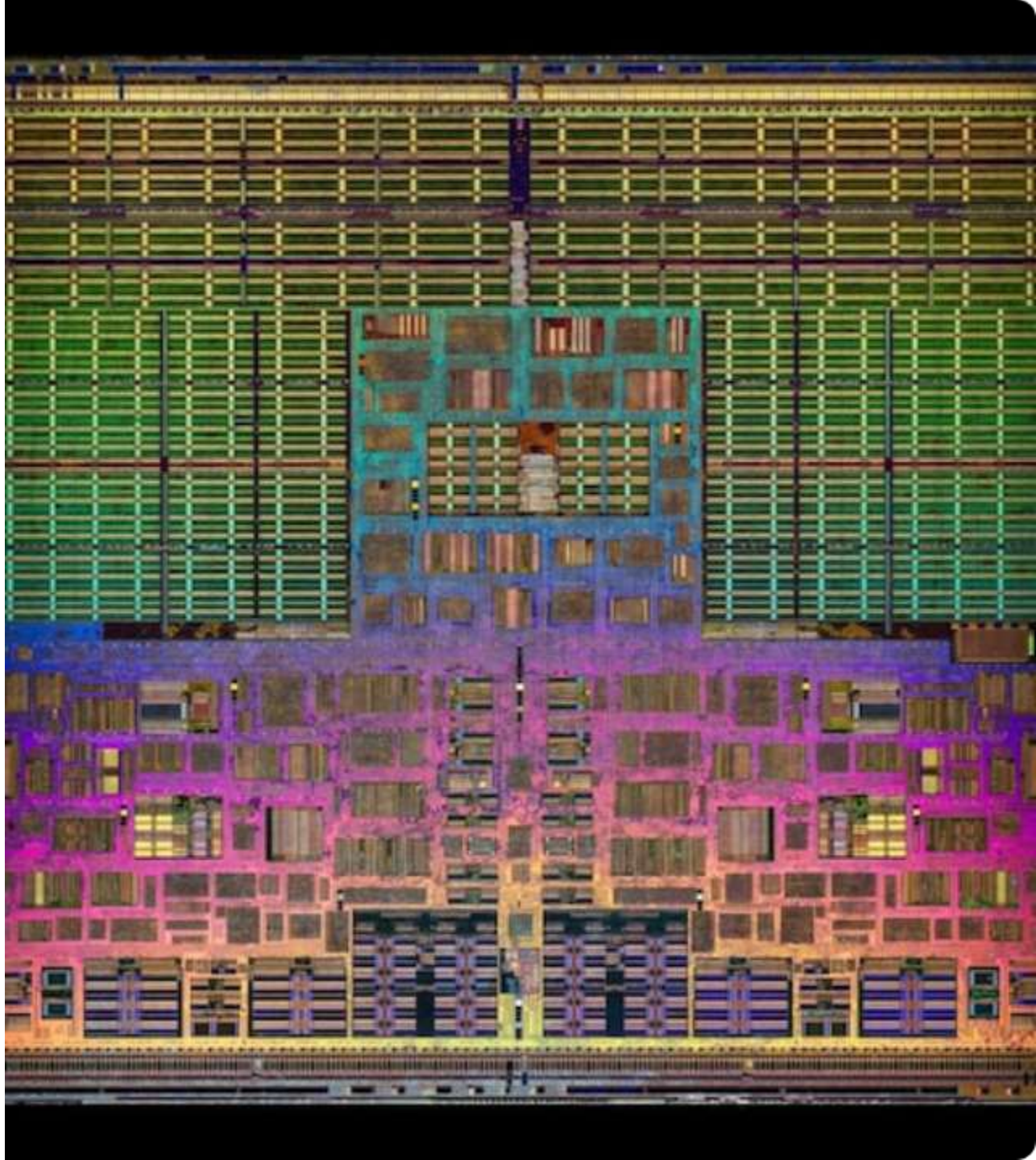
ISA在计算机系统中的核心地位

软硬件的纽带

ISA是上层软件（编译器、操作系统、应用程序）与下层硬件（CPU逻辑电路、微架构）之间的契约。

兼容性与演进

正因为有固定的ISA，高级语言编写的程序经编译为机器码后，可以在遵循同一ISA的不同CPU上直接运行，保证了软件生态的向前兼容。



指令的生命周期与执行过程

步骤 1

取指令 (IF)

根据程序计数器(PC)中的有效地址，从主存或Cache中取出当前指令存入指令寄存器(IR)，同时 PC 自动加 1 (或加指令字节数)。

步骤 2

指令译码 (ID)

控制器对 IR 中的操作码进行译码，识别指令操作类型，同时分析寻址方式，计算源操作数的有效地址(EA)。

步骤 3

执行指令 (EX)

根据译码结果，控制逻辑向相关部件（ALU、寄存器）发出微操作控制信号，完成指定的算术或逻辑运算。

步骤 4

访存与写回 (MEM/WB)

若需要读写主存数据，则进行主存访问；最后将运算结果写回指定的目标寄存器或存储单元中。

注意：指令执行过程中的微操作序列由 CPU 中的控制器（硬布线控制器或微程序控制器）统一发出时序信号驱动完成。

指令集的分类维度

按指令长度分类

- **定长指令字结构：**所有指令长度相等（如RISC指令通常定长4字节）。执行速度快，译码简单。
- **变长指令字结构：**指令长度不固定（如x86指令1~15字节）。占用空间小，但译码复杂。

按操作数物理位置分类

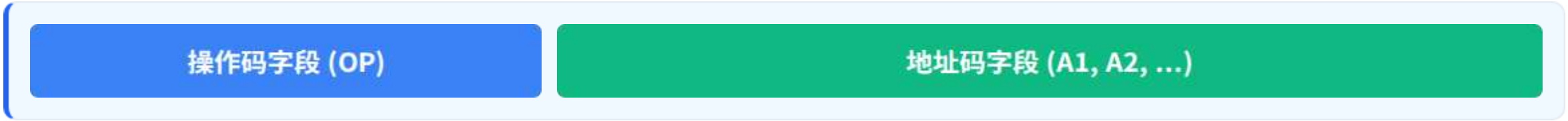
- **RR (Register-Register)：**操作数均在寄存器中，速度最快。
- **RS (Register-Storage)：**一个在寄存器，一个在主存。
- **SS (Storage-Storage)：**操作数均在主存中，访存次数多，效率低。

按功能类型分类

- **数据传送指令：**MOV, PUSH, POP
- **算术逻辑指令：**ADD, SUB, AND, OR
- **程序控制指令：**JMP, CALL, RET
- **I/O指令与特权指令：**IN, OUT

指令的基本结构：操作码 + 地址码

一条机器指令在逻辑上通常分为两大部分：



操作码 (Opcode, OP)

- **作用：**指定指令应完成何种操作（如加、减、传送、跳转等）。
- **定长操作码：**若操作码占 k 位，则最多能表示 2^k 条不同指令。
- **变长操作码：**操作码位数可变，常用扩展操作码技术。

地址码 (Address, A)

- **作用：**指定参与运算的操作数地址或运算结果的存储地址。
- **内容：**可以是主存地址、寄存器编号、或操作数本身（立即数）。
- **分类：**根据地址码个数可分为零、一、二、三、四地址指令。

指令操作码

操作码指出指令中该指令应该执行什么性质的操作和具有何种功能。

操作码是识别指令、了解指令功能与区分操作数地址内容的组成和使用方法等的关键信息。例如，指出是算术加运算，还是减运算；是程序转移，还是返回操作。

操作码分类：

定长操作码： 在指令字的最高位部分分配固定的若干位（定长）表示操作码。

- 一般n位操作码字段的指令系统最大能够表示2^n条指令。
- 优：定长操作码对于简化计算机硬件设计，提高指令译码和识别速度很有利；
- 缺：指令数量增加时会占用更多固定位，留给表示操作数地址的位数受限。

扩展操作码(不定长操作码)：全部指令的操作码字段的位数不固定，且分散地放在指令字的不同位置上。

- 最常见的变长操作码方法是扩展操作码，使操作码的长度随地址码的减少而增加，不同地址数的指令可以具有不同长度的操作码，从而在满足需要的前提下，有效地缩短指令字长。
- 优： 在指令字长有限的前提下仍保持比较丰富的指令种类；
- 缺 ：增加了指令译码和分析的难度，使控制器的设计复杂化。

按地址码数量分类

指令类型	指令格式结构	典型操作语义	主要应用场景
零地址指令	[OP]	不用地址码或隐含访问栈顶	停机(NOP)、堆栈指令(PUSH/POP)
一地址指令	[OP | A1]	(ACC) OP (A1) -> ACC	单操作数累加器指令、自增/自减
二地址指令	[OP | A1 | A2]	(A1) OP (A2) -> A1	最常用，双操作数算术/逻辑运算
三地址指令	[OP | A1 | A2 | A3]	(A1) OP (A2) -> A3	RISC架构常用，结果独立保存
四地址指令	[OP | A1 | A2 | A3 | A4]	(A1) OP (A2) -> A3; A4->PC	早期微程序控制，A4显式指定下条指令

注意：对于二地址指令，若地址码占 n 位，主存寻址范围为 2^n；若是寄存器寻址且有 M 个寄存器，则只需 $\log_2 M$位！

零地址与一地址指令机制

零地址指令 (Zero-Address)

1. **无操作数指令：**如空操作 NOP、停机 HALT、开中断 STI 等。
2. **堆栈计算机指令：**

- 操作数隐含保存在栈顶和次栈顶（从栈顶弹出两个数进行运算，结果压回栈顶）。
- 示例：ADD 指令的微操作：

$$(\text{栈顶}) + (\text{次栈顶}) \rightarrow \text{新栈顶}$$

一地址指令 (One-Address)

1. **单操作数指令：**如取反 NOT、求补、加 1、右移等，OP(A1) -> A1。
2. **隐含双操作数指令 (累加器ACC结构)：**

- 另一个操作数隐含保存在累加寄存器 ACC 中。
- 示例：ADD A1 指令微过程：

$$(\text{ACC}) + (A_1) \rightarrow \text{ACC}$$

- **访存次数：**取指1次 + 读操作数1次 = 2次访存。

二地址与三地址指令机制

二地址指令 (Two-Address)

- **格式：**[OP | A1(目的操作数) | A2(源操作数)]
- **运算逻辑：**(A1) OP (A2) -> A1
- **特点：**运算结果覆盖源目的地址 A1 原有内容。
- **访存分析 (主存寻址时)：**

取指1次 + 读A1(1次) + 读A2(1次) + 写A1(1次) = **共 4 次访存**。

三地址指令 (Three-Address)

- **格式：**[OP | A1 | A2 | A3(目的)]
- **运算逻辑：**(A1) OP (A2) -> A3
- **特点：**运算结果独立写入 A3，不破坏 A1 和 A2 数据。
- **访存分析 (主存寻址时)：**

取指1次 + 读A1(1次) + 读A2(1次) + 写A3(1次) = **共 4 次访存**。

408核心技巧：若二/三地址指令的操作数位于 CPU 内部通用寄存器中（如 ADD R1, R2），则不需要访问主存，访存次数降为取指的 1 次！

扩展操作码技术原理

设计初衷

在**定长指令字结构**中，为了在有限的指令字长内既能提供多种操作，又能支持不同数量的地址码，采用**操作码占用位数可变（扩展）**的编码方案。

核心编码规则 (前缀码原则)

1. **短操作码不能是长操作码的前缀：**确保译码的唯一性（类似于哈夫曼编码）。
2. **保留保留码 (Escape Code)：**当短操作码分配给某种指令后，必须留出至少一个编码组合作为向下一级扩展的标志。

典型扩展示意 (16位指令字)

- 4位OP + 三个4位地址码 (三地址指令)
- 8位OP + 两个4位地址码 (二地址指令)
- 12位OP + 一个4位地址码 (一地址指令)
- 16位OP + 无地址码 (零地址指令)

扩展操作码计算公式与递推推导

通用递推计算模型

假设指令字长固定为 N 位，每个地址码占 A 位：

$$\text{最高级可用编码总数} = 2^{\text{OP位数}}$$

若设三地址指令数为 M_3，则保留给二地址指令扩展的状态数为：

$$\text{保留状态数} = 2^{OP_3} - M_3$$

进而二地址指令的最大可能条数为：

$$M_{2,\max} = (2^{OP_3} - M_3) \times 2^A$$

寻址方式概念：指令寻址 vs 数据寻址

⏭1. 指令寻址 (Instruction Addressing)

- **目的：**寻找**下一条将要执行的指令**的主存地址。
- **实现依赖：**程序计数器 (PC)。
- **形式：**顺序寻址与跳跃寻址。

2. 数据寻址 (Data Addressing)

- **目的：**寻找**当前指令的操作数**真实有效地址 (EA, Effective Address)。
- **指令格式：**包含寻址特征位 (I/Mode) + 形式地址 (A)。

```
[ OP | 寻址特征 | 形式地址 A ]
```

概念区分：形式地址 A 是指令中显式写出的地址码；有效地址 EA 是最终访问主存或寄存器的实际物理地址。寻址方式即为 $A \rightarrow EA$ 的映射规则！

指令寻址：顺序寻址与跳跃寻址

顺序寻址 (Sequential)

- **逻辑：**按照指令在主存中的先后顺序依次执行。
- **PC 更新公式：**

$$(PC) + n \rightarrow PC$$

- 其中 n 为当前指令占用的**字节数**（按字节编址时）。若指令字长等于存储字长，且按字编址，则 $PC + 1 \rightarrow PC$。

跳跃寻址 (Jumping)

- **逻辑：**由转移类指令（如 JMP, CALL, BRANCH）改变程序执行流程。
- **PC 更新公式：**

$$\rightarrow PC$$

- 跳跃结果直接修改 PC 的内容，下条指令将从新的 EA 处取出。

数据寻址基础：10种寻址模式概览

10 种经典数据寻址方式：

基本寻址

1. 隐含寻址
2. 立即寻址
3. 直接寻址
4. 间接寻址

寄存器寻址

5. 寄存器寻址
6. 寄存器间接寻址

偏移量/复合寻址

7. 基址寻址
8. 变址寻址
9. 相对寻址
10. 堆栈寻址

重点**关注：**每种寻址方式的 **EA计算方法、是否访存及访存次数、典型使用场景**（如数组访问、多道程序浮动、栈帧维护等）。

基础寻址方式：隐含、立即、直接、间接

1. 隐含寻址 (Implicit)

- **特点：**指令中不显式给出操作数地址，隐含在专用寄存器（如 ACC）中。
- **优点：**缩短指令字长。

2. 立即寻址 (Immediate)

- **特点：**形式地址 A 即为操作数本身（补码形式，常记作 `#A`）。
- **EA：**无EA概念；**访存：**取指1次，指令执行阶段**0次访存**。

3. 直接寻址 (Direct)

- **公式：** $EA = A$，操作数 = (A)。
- **特点：**简单直观；**访存：**取指1次 + 读数据1次 = **2次访存**。

4. 间接寻址 (Indirect)

- **公式：** $EA = (A)$，操作数 = ((A))（可单次或多次间址）。
- **优点：**扩大寻址范围，便于编制子程序；**访存：**一次间址需 **3次访存**。

寄存器寻址与寄存器间接寻址

5. 寄存器寻址 (Register)

- **格式：**地址码给出寄存器编号 R_i。
- **有效地址/位置：**操作数在寄存器 R_i 中。

$$操作数 = (R_i)$$

- **特点：**不访问主存，仅访问CPU内部寄存器，执行速度极快！
- **访存次数：**仅取指 1 次。

6. 寄存器间接寻址 (Register Indirect)

- **格式：**地址码给出寄存器编号 R_i。
- **有效地址：**寄存器 R_i 中存放的是操作数的主存有效地址 EA。

$$EA = (R_i)，\quad 操作数 = ((R_i))$$

- **优点：**比主存间接寻址快得多，只需访问主存 1 次数据。
- **访存次数：**取指1次 + 读主存1次 = **2次访存**。

偏移量寻址(1)：基址寻址 (Base-Register)

基本原理与算式

将 CPU 中**基址寄存器 BR (Base Register)** 的内容加上指令中的形式地址 A 形成有效地址 EA。

$$EA = (BR) + A$$

（注：BR 也可以是指定的通用寄存器）

应用场景与核心特征

- **主要应用：**多道程序设计、操作系统动态重定位（逻辑地址转物理地址）。
- **程序员权限：BR内容由操作系统管理/修改**，对用户程序员透明（不可见）；形式地址 A 可由程序员指定。
- **优点：**可扩大寻址范围，有利于程序浮动。

偏移量寻址(2)：变址寻址 (Index-Register)

基本原理与算式

将 CPU 中**变址寄存器 IX (Index Register)** 的内容加上形式地址 A 形成有效地址 EA。

$$= (IX) + A$$

应用场景与对比辨析

- **主要应用：**数组遍历、循环程序设计。形式地址 A 作为数组首地址，变址寄存器 IX 内容作为数组下标偏移量。
- **程序员权限：IX 的内容由用户程序修改**，形式地址 A 在循环中保持不变。
- **复合变址：**变址基址重叠寻址 $EA = (BR) + (IX) + A$。

辨析：基址寻址中“基址变，偏移不变”；变址寻址中“基址(A)不变，偏移(IX)变”！

偏移量寻址(3)：相对寻址与PC更新细节

基本原理与算式

把程序计数器 PC 的当前内容加上指令格式中的形式地址 A（位移量，补码形式）形成有效地址：

$$= (PC) + A$$

⚠陷阱：PC的自动加值

在取指阶段，CPU 读出当前指令后，**PC 已自动加上当前指令的字节数 L**!

所以在计算跳转有效地址时，使用的 (PC) 是**下一条指令的地址**:

$$= PC_{当前} + L + A$$

主要特点与应用场景

- **典型应用：**转移指令（如 JMP, JZ）、程序浮动与模块化代码。
- **优点：**无论程序装入主存的什么位置，指令与目标地址之间的相对位移量 A 保持不变。

堆栈寻址方式 (Stack Addressing)

堆栈类型与特点

- **硬堆栈：**由CPU内部一组寄存器组成，速度极快，容量有限。
- **软堆栈：**在主存中划出一块专门区域作为堆栈（x86采用），由栈指针寄存器 SP (Stack Pointer) 指向栈顶。

进栈 (PUSH) 与 出栈 (POP) 操作 (以压栈为例)

假设堆栈向低地址生长，字长 4 字节：

1. **PUSH 操作微过程：**

$$(SP) - 4 \rightarrow SP;\quad 数据 \rightarrow M[SP]$$

2. **POP 操作微过程：**

$$M[SP] \rightarrow 目标;\quad (SP) + 4 \rightarrow SP$$

【汇总】十大数据寻址方式全方位对比

03 总结对比

寻址方式	有效地址 EA 算法	访存次数 (指令执行期)	主要优点 / 典型应用场景
立即寻址	无EA (A即操作数)	0 次	速度最快；给变量赋初值
直接寻址	EA = A	1 次	简单；寻找静态全局变量
间接寻址	EA = (A)	2 次 (或多次)	扩大寻址范围；指针间接引用
寄存器寻址	无主存EA (在寄存器)	0 次	速度极快；高频局部变量
寄存器间接	$EA = (R_i)$	1 次	比主存间址快；指针与连续访存
基址寻址	$EA = (BR) + A$	1 次	操作系统动态重定位；程序浮动
变址寻址	$EA = (IX) + A$	1 次	数组遍历、循环结构
相对寻址	$EA = (PC) + A$	1 次	转移指令、浮动代码编写

数据的边界对齐 (Data Alignment)

对齐规则 (Aligned)

- **核心要求：**数据在主存中的存放起始地址必须是该数据类型所占字节数的整数倍。
- **例如 (在32位机器上)：**

- char (1B)：可存放在任意字节地址。
- short (2B)：起始地址必须是 2 的倍数（最低二进制位为0）。
- int/float (4B)：起始地址必须是 4 的倍数（最低2位为00）。

为什么需要边界对齐?

- 存储器通常按**存储字（如4字节/8字节）**进行读取。
- 若数据未对齐，跨越了两个存储字边界，CPU 访问该数据需要**两次读主存周期**，并进行内部拼接！
- 边界对齐是以“**空间换时间**”提高 CPU 访存效率。

↔ 大端方式与小端方式 (Byte Order)

当多字节数据（如 32 位整数 0x12345678）写入按字节编址的主存时，字节的排列顺序有两种标准：

大端方式 (Big-Endian)

- **核心口诀：高位字节存低地址，低位字节存高地址。**
- 符合人类阅读习惯（如网络协议字节序）。

```
地址：  0x00(12)  |  0x01(34)  |  0x02(56)  |  0x03(78)
```

小端方式 (Little-Endian)

- **核心口诀：低位字节存低地址，高位字节存高地址。**
- 符合计算机硬件加法运算逻辑（如 x86 架构）。

```
地址：  0x00(78)  |  0x01(56)  |  0x02(34)  |  0x03(12)
```

CISC (复杂指令集计算机)

设计理念

- **背景：**早期主存容量小且昂贵，编译器技术不够成熟，为了“缩短程序长度”而不断增加复杂的硬件指令。
- **典型代表：**x86 架构 (Intel Core, AMD Ryzen)。

主要技术特征

- 指令系统庞大，指令条数多（数百条）。
- 指令长度不固定，指令格式与寻址方式极多。
- 寻址方式复杂，许多指令可以直接访问主存。
- 各种指令执行时间相差巨大，CPI 远远大于 1。
- 控制器多采用**微程序控制器**实现。

RISC (精简指令集计算机)

设计理念

- **80/20规律：**统计表明 80% 的程序只频繁使用 20% 的简单指令！
- 优先优化高频指令，通过高度流水线化大幅提高 CPU 频率。
- **典型代表：**ARM, RISC-V, MIPS。

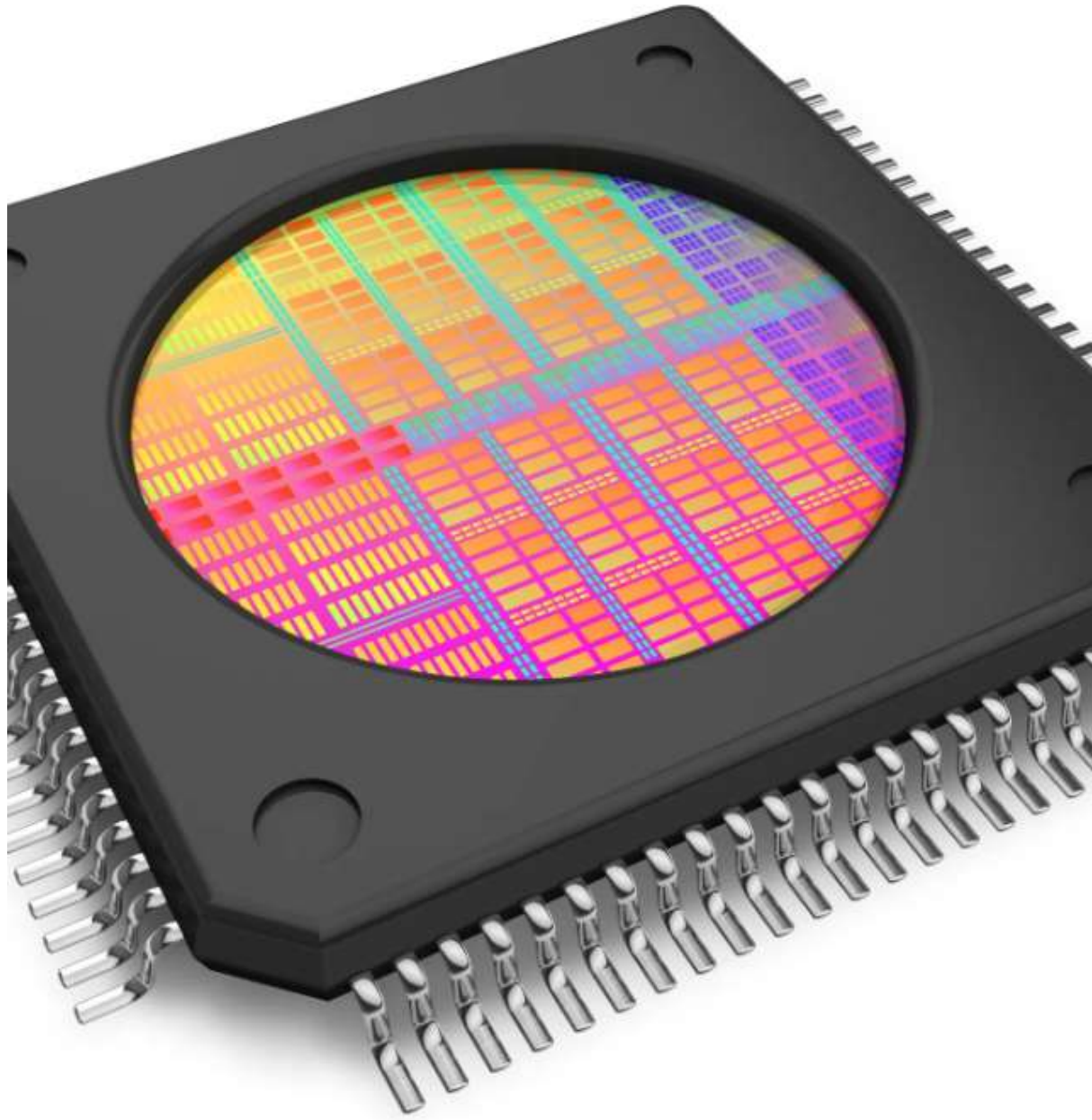
主要技术特征

- 选取使用频率最高的一组简单指令，格式规整。
- 指令定长（通常为32位），寻址方式少。
- **只有 Load/Store 指令能访存**，其余运算均在通用寄存器之间进行。
- 通用寄存器数量多，大多数指令在 1 个时钟周期内完成。
- 控制器采用**硬布线逻辑**，极易实现指令流水线。

⚖ CISC 与 RISC 全方位对比

融合演进趋势

现代高性能 CPU 吸收了两者的优点：如 Intel x86 处理器在内部前端将复杂的 CISC 指令解码转换为定长的 RISC 风格微操作 (uops) 在流水线中执行！



CISC 与 RISC 核心特征对照表

05 对比总结

对比维度	CISC (复杂指令集)	RISC (精简指令集)
指令条数与格式	指令多、变字长、格式不固定	指令少、定字长、格式规范统一
访存指令限制	无限制，多种指令可直接访存	**严格限制只有 Load/Store 可访存**
通用寄存器数量	相对较少 (如早期x86仅8个)	多 (通常32个或更多)
目标代码长度	较短（编码紧凑）	较长（需要更多指令）
指令周期与CPI	差异大，CPI >> 1	绝大多数1周期完成，CPI ≈ 1
控制器实现	主要采用微程序控制器	主要采用硬布线控制器（组合逻辑）
流水线优化	流水线实现难度大	极为适合指令流水线技术

x86-64 架构常用通用寄存器及其约定

返回值与特殊寄存器

- %rax：返回值，也用于乘除法运算。
- %rsp：栈指针寄存器 (Stack Pointer)，指向栈顶。
- %rbp：帧指针寄存器 (Frame Pointer)，指向当前栈帧基址。

函数调用参数寄存器

x86-64 规定前 6 个整数/指针参数依次使用：

1. %rdi (第1参数)
2. %rsi (第2参数)
3. %rdx (第3参数)
4. %rcx (第4参数)
5. %r8 (第5参数)，6. %r9 (第6参数)

寄存器保存责任划分

- **调用者保存 (Caller-saved)：**%rax, %rdi, %rsi, %rdx, %rcx, %r8~%r11（若主调函数后续还要用，需在call前压栈）。
- **被调用者保存 (Callee-saved)：**%rbx, %rbp, %r12~%r15（被调函数若要修改，必须先压栈备份，返回前恢复）。

x86-64 核心汇编指令速查

指令类型	典型汇编格式 (AT&T格式)	操作语义说明
数据传送	movq %rax, %rbx	%rax -> %rbx (传送 8 字节)
加载有效地址	leaq 8(%rdx, %rcx, 4), %rax	计算 %rdx + %rcx*4 + 8 -> %rax (不访存)
算术运算	addq %rax, %rbx / subq	%rbx + %rax -> %rbx，设置条件码
比较与测试	cmpq %rax, %rbx / testq	计算 %rbx - %rax，仅设置条件码，不保存结果
无条件跳转	jmp .L1	无条件跳转到目标标签 .L1 处
条件跳转	je / jne / jl / jge .L1	根据标志位 (ZF, SF, OF等) 条件跳转

AT&T格式 v.s. Intel格式

	AT&T 格式	Intel 格式
目的操作数d、源操作数s	op s, d 注：源操作数在左，目的操作数在右	op d, s 注：源操作数在右，目的操作数在左
寄存器的表示	mov %ebx, %eax 注：寄存器名之前必须加"%"	mov eax, ebx 注：直接写寄存器名即可
立即数的表示	mov $985, %eax 注：立即数之前必须加“$”	mov eax, 985 注：直接写数字即可
主存地址的表示	mov %eax , (af996h) 注：用“小括号”	mov [af996h], eax 注：用“中括号”
读写长度的表示	movb $5, (af996h) movw $5, (af996h) movl $5, (af996h) addb $4, (af996h) 注：指令后加 b、w、l 分别表示读写长度为byte、word、dword	mov byte ptr [af996h], 5 mov word ptr [af996h], 5 mov dword ptr [af996h], 5 add byte ptr [af996h], 4 注：在主存地址前说明读写长度byte、word、dword
主存地址偏移量的表示	movl -8(%ebx), %eax *注：偏移量(基址)* movl 4(%ebx, %ecx, 32), %eax *注：偏移量(基址,变址,比例因子)*	mov eax, [ebx - 8] *注：[基址+偏移量]* mov eax, [ebx + ecx*32 + 4] *注：[基址+变址*比例因子+偏移量]*

EFLAGS 条件码与分支预测性能

零标志位 ZF (Zero)

若最近运算结果为 0，则 ZF=1；否则 ZF=0。

je (等于时跳转) 检查 ZF=1。

符号标志位 SF (Sign)

记录最近运算结果的最高符号位。若为负数则 SF=1。

溢出标志位 OF (Overflow)

若有符号数运算发生溢出，则 OF=1。

有符号数比较 x < y 条件为：$SF \oplus OF = 1$!

⚡条件传送指令 (cmov) 优化

为了避免条件跳转引发的 CPU **流水线分支预测失败（惩罚耗时大）**，编译器常将简单的 if-else 编译为条件传送指令（如 cmovge %rdx, %rax），使分支无需跳转，保持流水线顺畅!

3. 循环结构语句的机器级表示

高级语言中的 do-while、while 和 for 循环，在机器级汇编中均通过“**条件比较 + 条件跳转**”构造：

do-while 结构

直接先执行循环体，再在末尾进行条件判断，若满足则跳转回循环头部。

while 结构 (Jump-to-middle)

首次直接无条件跳转到末尾的条件测试代码处，测试通过再跳回循环体。

for 结构

先初始化变量，随后转化为 while 或 do-while 的结构进行跳转控制。

4. 过程(函数)调用的栈帧结构 (Stack Frame)

栈帧 (Stack Frame) 定义

- 每一个正在调用的函数在主存栈区中都拥有独立的一块区域，称为**栈帧**。
- 栈顶指针 %rsp：指向栈顶元素。
- 栈基址指针 %rbp：指向当前栈帧的底部。
- 栈向**低地址方向生长**。

栈帧内部包含的内容 (由高地址到低地址)

1. 函数调用参数 (超过6个的部分，第7+个参数)
2. **返回地址 (Return Address)**（由 call 压入）
3. 保存的旧 %rbp（由被调函数压入）
4. 被调用者保存的寄存器备份
5. 局部变量与临时缓冲区

栈帧分配与运行时内存

栈帧保护机制

过程调用通过精确维护 %rsp 和 %rbp 指针，确保子函数退出时能彻底恢复主调函数的寄存器状态与执行上下文。



call 与 ret 指令微操作与 PC 变化

call 指令微操作 (调用函数)

执行 call Label 时，CPU 完成以下两步动作：

1. 将 call 的下一条指令地址（即返回地址）压入栈中：

$$(\text{rsp}) - 8 \rightarrow \text{rsp};\quad \text{PC} \rightarrow \text{M}[\text{rsp}]$$

2. 将目标函数入口地址装入 PC：

$$\text{Label} \rightarrow \text{PC}$$

ret 指令微操作 (过程返回)

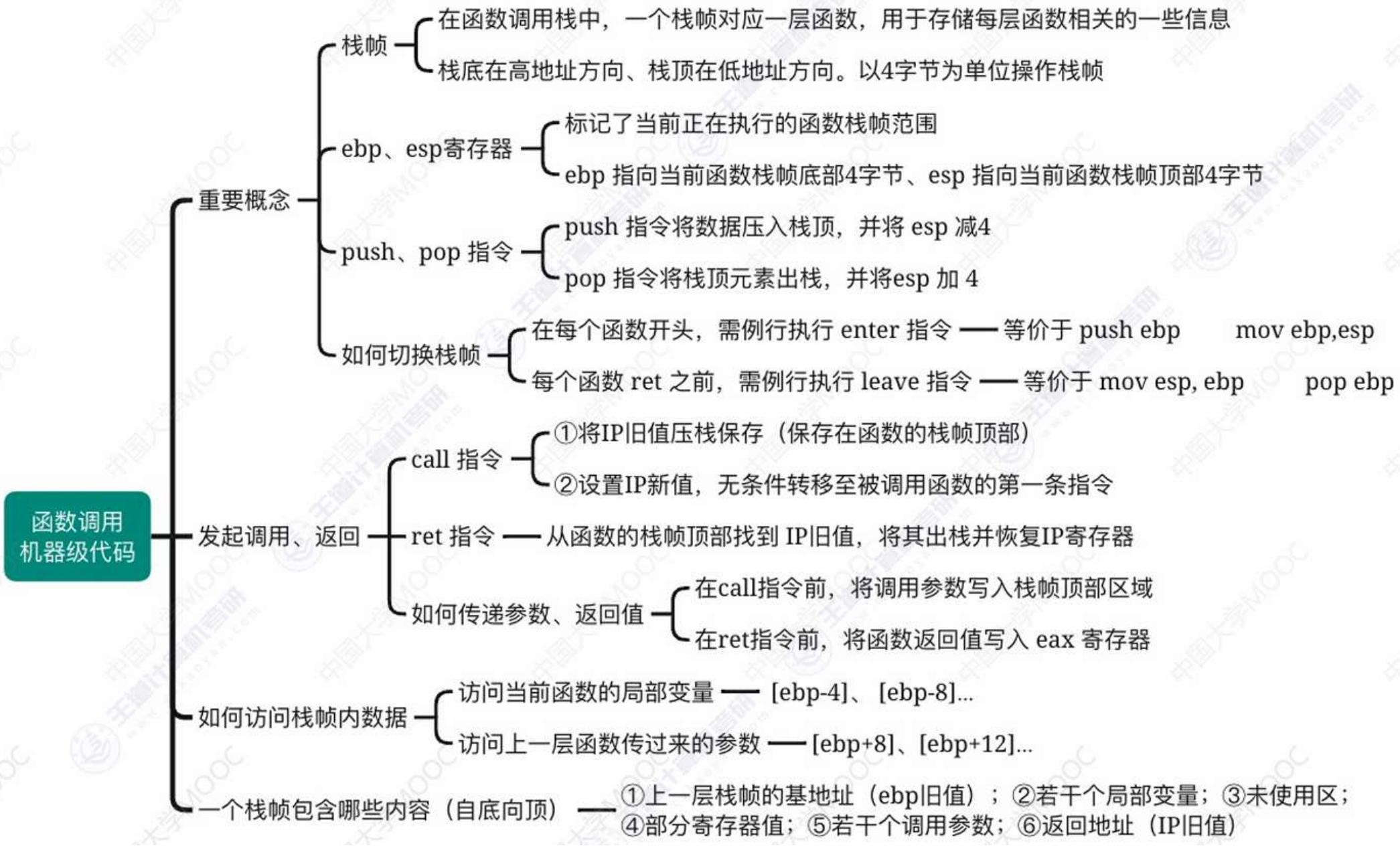
执行 ret 时，CPU 弹出栈顶的返回地址装回 PC：

1. 从栈顶弹出返回地址给 PC：

$$\text{M}[\text{rsp}] \rightarrow \text{PC}$$

2. 恢复栈顶指针 %rsp：

$$(\text{rsp}) + 8 \rightarrow \text{rsp}$$


函数调用
机器级代码
重要概念
栈帧
在函数调用栈中，一个栈帧对应一层函数，用于存储每层函数相关的一些信息
栈底在高地址方向、栈顶在低地址方向。以4字节为单位操作栈帧
ebp、esp寄存器
标记了当前正在执行的函数栈帧范围
ebp 指向当前函数栈帧底部4字节、esp 指向当前函数栈帧顶部4字节
push、pop 指令
push 指令将数据压入栈顶，并将 esp 减4
pop 指令将栈顶元素出栈，并将esp 加 4
如何切换栈帧
在每个函数开头，需例行执行 enter 指令 —— 等价于 push ebp mov ebp,esp
每个函数 ret 之前，需例行执行 leave 指令 —— 等价于 mov esp, ebp pop ebp
发起调用、返回
call 指令
①将IP旧值压栈保存（保存在函数的栈帧顶部）
②设置IP新值，无条件转移至被调用函数的第一条指令
ret 指令 —— 从函数的栈帧顶部找到 IP旧值，将其出栈并恢复IP寄存器
如何传递参数、返回值
在call指令前，将调用参数写入栈帧顶部区域
在ret指令前，将函数返回值写入 eax 寄存器
如何访问栈帧内数据
访问当前函数的局部变量 —— [ebp-4]、[ebp-8]...
访问上一层函数传过来的参数 —— [ebp+8]、[ebp+12]...
一个栈帧包含哪些内容（自底向顶） ——
①上一层栈帧的基地址（ebp旧值）；②若干个局部变量；③未使用区；
④部分寄存器值；⑤若干个调用参数；⑥返回地址（IP旧值）

17.【2022 统考真题】设计某指令系统时，假设采用 16 位定长指令字格式，操作码使用扩展编码方式，地址码为 6 位，包含零地址、一地址和二地址 3 种格式的指令。若二地址指令有 12 条，一地址指令有 254 条，则零地址指令的条数最多为（ ）。

A. 0　　B. 2　　C. 64　　D. 128

17. D

地址码为 6 位，一条二地址指令会占用 2^6 条一地址指令的空间，一条一地址指令会占用 2^6 条零地址指令的空间。若全都是零地址指令，则最多有 2^{16} 条，减去一地址指令和二地址指令所占用的零地址指令空间，即 $2^{16}-254\times2^6-12\times2^6\times2^6=(2^{10}-254-12\times2^6)\times2^6=2\times2^6=128$。

另解：二地址指令有 12 条，则剩余 16 − 12 = 4 种操作码给一地址指令，一地址指令有 254 条，剩余 $4\times64-254=2$ 种操作码给 0 地址指令，所以 0 地址一共有 $2\times2^6=128$ 条。

32.【2019 统考真题】某计算机采用大端方式，按字节编址。某指令中操作数的机器数为 1234 FF00H，该操作数采用基址寻址方式，形式地址（用补码表示）为 FF12H，基址寄存器的内容为 F000 0000H，则该操作数的 LSB（最低有效字节）所在的地址是（ ）。

A. F000 FF12H　　B. F000 FF15H　　C. EFFF FF12H　　D. EFFF FF15H

32. D

注意，内存地址是无符号数。

操作数采用基址寻址方式，EA = (BR) + A，基址寄存器 BR 的内容为 F000 0000H，形式地址用补码表示为 FF12H 即 1111 1111 0001 0010B，因此有效地址为 F000 0000H + (−00EEH) = EFFF FF12H。计算机采用大端方式编址，所以低位字节存放在字的高地址处，机器数一共占 4 字节，该操作数的 LSB 所在的地址是 EFFF FF12H + 3 = EFFF FF15H。

33.【2020 统考真题】某计算机采用 16 位定长指令字格式，操作码位数和寻址方式位数固定，指令系统有 48 条指令，支持直接、间接、立即、相对 4 种寻址方式。在单地址指令中，直接寻址方式的可寻址范围是（ ）。

A. 0～255　　B. 0～1023　　C. −128～127　　D. −512～511

33. A

48 条指令需要 6 位操作码字段（$2^5 < 48 < 2^6$），4 种寻址方式需要 2 位寻址特征位（$4 = 2^2$），还剩 $16-6-2=8$ 位作为地址码，所以直接寻址范围为 0～255。注意，主存地址不能为负。

34.【2023 统考真题】某运算类指令中有一个地址码为通用寄存器编号，对应通用寄存器中存放的是操作数或操作数的地址，CPU 区分两者的依据是（　）。

A. 操作数的寻址方式　　B. 操作数的编码方式

C. 通用寄存器的编号　　D. 通用寄存器的内容

34. A

指令字由操作码、寻址特征和地址码三个字段组成，寻址特征字段用来指明指令属于哪种寻址方式。若寻址方式是寄存器直接寻址，则地址码所指的通用寄存器中存放的是操作数，若寻址方式是寄存器间接寻址，则对应通用寄存器中存放的是操作数的地址。

12. 假设 P 为调用过程，Q 为被调用过程，程序在 32 位 x86 处理器上执行，以下是 C 语言程序中过程调用所涉及的操作：

① 过程 Q 保存 P 的现场，并为非静态局部变量分配空间

② 过程 P 将实参存放到 Q 能访问到的地方

③ 过程 P 将返回地址存放到特定处，并转跳到 Q 执行

④ 过程 Q 取出返回地址，并转跳回到过程 P 执行

⑤ 过程 Q 恢复 P 的现场，并释放局部变量所占空间

⑥ 执行过程 Q 的函数体

过程调用的正确执行步骤是（ ）。

A. ②→③→④→①→⑤→⑥　　B. ②→③→①→④→⑥→⑤

C. ②→③→①→⑥→⑤→④　　D. ②→③→①→⑤→⑥→④

12. C

过程调用的具体过程已在 4.3.4 节中介绍。

05. 【2009 统考真题】下列关于 RISC 的说法中，错误的是（ ）。

A. RISC 普遍采用微程序控制器

B. RISC 大多数指令在一个时钟周期内完成

C. RISC 的内部通用寄存器数量相对 CISC 多

D. RISC 的指令数、寻址方式和指令格式种类相对 CISC 少

05. A

相对于 CISC，RISC 的特点是：指令条数少；指令长度固定，指令格式和寻址种类少；只有取数/存数指令访问存储器，其余指令的操作均在寄存器之间进行；CPU 中通用寄存器多；大部分指令在一个时钟周期内完成；以硬布线逻辑为主，不用或少用微程序控制。B、C 和 D 都是 RISC 的特点。由于 RISC 的速度快，因此普遍采用硬布线控制器，A 错误。

某计算机指令字长为 16 位，系统中有 16 个通用寄存器。指令集中包含三地址指令、二地址指令、一地址指令和零地址指令。所有寄存器操作数均在指令中显式给出（即每个寄存器地址码占 4 位）。已知系统目前需要支持 14 条三地址指令，X 条二地址指令，50 条一地址指令，以及 64 条零地址指令。在保证前缀扩展操作码编码合法的情况下，X 的最大值是（ ）

A. 28
B. 29
C. 30
D. 31

【答案】：A

【解析】：三地址指令：格式为 OP(4位) | R1(4位) | R2(4位) | R3(4位)。4 位操作码最多可编码 16 条指令。现需要 14 条三地址指令，因此剩下 $16-14=2$个操作码作为下一级扩展的前缀。

二地址指令：格式为 OP(8位) | R1(4位) | R2(4位)。由三地址剩余的 2 个前缀扩展而来，最大空间为 $2\times 2^4=32$条。假设使用了 X条，则剩余 $32-X$个前缀用于继续扩展。

一地址指令：格式为 OP(12位) | R1(4位)。由二地址剩余的 $32-X$个前缀扩展，最大空间为 $(32-X)\times 2^4=(32-X)\times 16$条。系统中需支持 50 条一地址指令，因此一地址剩余的前缀数为 $(32-X)\times 16-50$。

零地址指令：格式为 OP(16位)。由一地址剩余的前缀扩展，最大空间为 $[(32-X)\times 16-50]\times 2^4$条。系统中需支持 64 条零地址指令，故必须满足：

$$[(32-X)\times 16-50]\times 16\geq 64$$

$$(32-X)\times 16-50\geq 4$$

$$(32-X)\times 16\geq 54$$

$$32-X\geq 3.375$$

因为 X必须为整数，所以 $32-X\geq 4$，解得 $X\leq 28$。当 $X=28$时，一地址指令空间有 $4\times 16=64$条（分配 50 条后剩 14 个前缀），零地址指令空间有 $14\times 16=224$条，满足 64 条的需求。

某 32 位 CPU 使用补码表示带符号整数，其 ALU 运行一条比较指令 CMP R1，R2（内部执行 R1 − R2 的减法操作，不保存结果，仅影响标志位）。执行后，标志寄存器中的状态被置为：SF=0，OF=1，CF=0，ZF=0。根据此时的标志位状态，关于 R1 和 R2 中存放的 32 位二进制数的大小关系，下列判断正确的是（ ）

A. 若视为有符号数，R1 < R2；若视为无符号数，R1 > R2
B. 若视为有符号数，R1 > R2；若视为无符号数，R1 < R2
C. 若视为有符号数，R1 < R2；若视为无符号数，R1 < R2
D. 若视为有符号数，R1 > R2；若视为无符号数，R1 > R2

【答案】：A
【解析】：**有符号数比较**：比较 R1 和 R2 时，若判定 R1 < R2，对应的逻辑条件是 SF ⊕ OF = 1（即 SF 与 OF 不等）。题目中 SF=0（结果为正），OF=1（发生溢出）。正溢出意味着“负数减去正数得到了正结果”，即 R1 本身为负，R2 为正，真实数学结果为负（R1 < R2），但超出了表示范围导致符号位翻转变成 0。因为 0 ⊕ 1 = 1，所以对于有符号数，结论是 R1 < R2。
无符号数比较：无符号数的减法使用进位/借位标志 CF。在 R1 − R2 中，若 CF=1，表示发生了借位，即无符号的 R1 < R2；若 CF=0，表示未发生借位，即无符号的 R1 ≥R2。题目中 CF=0，且 ZF=0（结果不为零，即 R1 ≠R2），因此对于无符号数，结论是 R1 > R2。

某 32 位系统按字节编址，采用小端方式（Little-Endian）存储，栈向低地址方向增长。当前栈指针 ESP 的值为 0x0010 0020。PC 当前正指向地址 0x0800 1000，此处存放着一条 5 字节长的过程调用指令 CALL 0x0800 2000。 CPU 取出并执行该 CALL 指令后，进入子程序执行的第一条指令是 1 字节长的 PUSH EBP。在这两条指令均执行完毕后，ESP 的值以及物理内存地址 0x0010 001D 处存放的字节内容分别是（ ）

A. 0x0010 0018， 0x10
B. 0x0010 001C， 0x10
C. 0x0010 0018， 0x00
D. 0x0010 001C， 0x08

【答案】：A
【解析】：**CALL 指令执行**：CALL 指令长 5 字节，取指完成后，PC 自动指向下一条指令，即返回地址 = 0x0800 1000 + 5 = 0x0800 1005。执行 CALL 会将该 32 位返回地址压栈。栈指针 ESP 减 4，变为 0x0010 0020 − 4 = 0x0010 001C。
小端存储分析：返回地址 0x0800 1005 存入从 0x0010 001C 开始的 4 个连续字节中。按小端规则，低有效字节存在低地址：

0x0010 001C 存放 0x05
0x0010 001D 存放 0x10
0x0010 001E 存放 0x00
0x0010 001F 存放 0x08

PUSH 指令执行：子程序执行 PUSH EBP，将 32 位的 EBP 寄存器压栈。ESP 再次减 4，变为 0x0010 001C − 4 = 0x0010 0018。
综上，最终 ESP 为 0x0010 0018，地址 0x0010 001D 处的字节为 0x10。

某 32 位计算机采用类似 MIPS 的指令架构，按字节编址。指令集中包含两种控制流指令：

- **J 型指令（无条件跳转）**：高 6 位为操作码，低 26 位为立即数。目标地址计算方式为：取（PC+4）的最高 4 位，拼接上 26 位立即数左移 2 位后的结果。
- **I 型指令（条件转移）**：高 6 位为操作码，随后两个 5 位寄存器，低 16 位为立即数（补码表示）。目标地址采用 PC 相对寻址：PC+4 +（16位立即数符号扩展后左移 2 位）。

已知物理内存中 0x0040 0000 处存放着一条 J 型指令，其机器码为 0x0810 0005；0x0040 0010 处存放着一条 I 型条件转移指令 BEQ，其机器码为 0x1000 FFFC。若这两条指令均发生跳转，其目标地址分别是（ ）

A. 0x0040 0014， 0x0040 0014
B. 0x0400 0014， 0x0040 0004
C. 0x0400 0014， 0x003F FFFC
D. 0x0040 0014， 0x0040 0004

【答案】：B
【解析】：J 型指令目标计算： 机器码 0x0810 0005 的二进制前 6 位 0000 10 为操作码，后 26 位为 00 0001 0000 0000 0000 0000 0101，即十六进制 0x100005。 当前 PC = 0x0040 0000，PC+4 = 0x0040 0004，其最高 4 位为 0（0000）。 26 位立即数左移 2 位（等价于乘 4）得：0x100005 << 2 = 0x400014。拼接最高 4 位后，目标地址为 0x0400 0014。
I 型指令（BEQ）目标计算： 机器码 0x1000 FFFC 的低 16 位立即数为 0xFFFC。由于是补码，它代表十进制的 -4。 当前 PC = 0x0040 0010，PC+4 = 0x0040 0014。立即数符号扩展后左移 2 位（即 -4 * 4 = -16），对应的十六进制偏移量为 -0x10。目标地址 = 0x0040 0014 - 0x10 = 0x0040 0004。

给定 8 位寄存器 AL 的初始值为 0x8A。CPU 对该寄存器按顺序执行以下三条机器指令：

1. SAR AL, 1 （算术右移 1 位）
2. ROL AL, 1 （不带进位位的循环左移 1 位）
3. SHR AL, 1 （逻辑右移 1 位）

在上述三条指令全部执行完毕后，寄存器 AL 的值以及进位标志位 CF 的状态分别是（ ）

A. 0x45， CF = 1
B. 0xC5， CF = 0
C. 0x45， CF = 0
D. 0x8B， CF = 1

【答案】：A
【解析】：初始 AL = 0x8A = 1000 1010（二进制）。
SAR AL, 1（算术右移）：向右移 1 位，最高位（符号位）保持不变，移出的最低位进入 CF。 原最高位为 1，右移后补 1，得到 1100 0101，即十六进制 0xC5。移出的最低位是 0，此时 CF = 0。
ROL AL, 1（循环左移）：整体向左移 1 位，最高位不仅移入最低位，同时也复制到 CF 标志中。 对 1100 0101 左移 1 位：最高位 1 移入最低位，其余位左移，得到 1000 1011，即十六进制 0x8B。最高位 1 进入 CF，此时 CF = 1。
SHR AL, 1（逻辑右移）：向右移 1 位，最高位强制补 0，移出的最低位进入 CF。 对 1000 1011 右移 1 位：最高位补 0，得到 0100 0101，即十六进制 0x45。移出的最低位是 1，此时 CF = 1。
最终，AL = 0x45，CF = 1。